

Importance of Design Patterns

1. Improve Flexibility and Scalability

Design patterns make software easier to extend and adapt when requirements change. They allow adding new features with minimal modification to existing code.

2. Improve Maintainability and Reduce Complexity

Design patterns organize code into clear structures with proper responsibilities. They make debugging, updating, and managing large systems easier.

3. Support Refactoring and Remove Code Smells

Design patterns help improve existing code by replacing poor designs with better solutions. They reduce issues like duplicate code, large conditional statements, and tight coupling.

4. Increase Code Reusability

Design patterns provide proven solutions that can be reused in different projects. They reduce development time and prevent rewriting common design logic.

5. Improve Testing and Code Quality

Design patterns create loosely coupled and modular code that is easier to test. They allow individual components to be tested independently, improving reliability.

Category of Design Pattern and Their Purpose

Design patterns are reusable solutions to common software design problems. There are three types of Design Patterns

1. Creational Design Patterns

Creational patterns deal with object creation mechanisms. They hide the object creation process and make the system independent of how objects are created, composed, and represented

Main Problems Solved:

- Avoid complex object creation logic.
- Reduce dependency between client code and concrete classes.

1. Singleton Pattern

Purpose:

- Ensures that only one instance of a class exists.
- Provides a global access point to that instance.

Advantages

- Ensures only one instance exists, preventing inconsistent states.
- Saves memory and system resources by avoiding multiple object creation.
- Provides a global access point, making it easy to share the same instance across the application.

Disadvantages

Despite its benefits, Singleton can introduce certain design challenges:

- Can create tight coupling since many classes depend on a global instance.
- Difficult to unit test due to shared global state.
- May cause issues in multithreaded environments if not implemented carefully.

2. Factory Pattern

Purpose:

- Creates objects without exposing the object creation logic to the client.
- Allows object creation decisions at runtime.

Advantages

The Factory Method pattern provides flexibility and better design structure in object creation:

- Encapsulates object creation logic, so clients don't need to know how objects are created internally.
- Promotes loose coupling by depending on abstractions instead of concrete classes.
- Supports scalability and extensibility by allowing new product types without modifying existing code.

Disadvantages

Although useful, the pattern can introduce some complexity:

- Increases the number of classes in the system.
- Can make the code more complex compared to simple object creation.
- May be unnecessary for small or simple applications.

3. Abstract Factory Pattern

Purpose:

- Creates families of related objects without specifying their concrete classes.
- Provides a common interface for creating multiple related products.

Usage:

- Creating related objects such as teachers, officers, and staff.

Advantages

The Abstract Factory Pattern improves modularity, flexibility, and scalability in large systems.

- Promotes **loose coupling** between client and product classes.
- Makes switching product families easy without modifying client code.
- Supports **Open/Closed** Principle.

Disadvantages

Although powerful, Abstract Factory can introduce complexity and additional overhead.

- Increases number of classes and interfaces.
- Can make the system more complex than necessary for small projects.
- Adding a new product type requires modifying the abstract factory interface

4. Builder Pattern

Purpose:

- Creates complex objects step by step.
- Separates object construction from object representation.

Usage:

- Creating objects with many attributes.

Advantages

- Builds complex objects step by step using a separate Builder class, improving control over object creation.
- Separates **construction logic** from representation, allowing flexible and reusable object creation.
- Improves readability by avoiding **large constructors** and can promote immutability of the final object.

Disadvantages

- Increases the number of classes in the system
- Adds unnecessary complexity for simple objects
- Requires changes if the product structure changes significantly

2. Structural Design Patterns

Structural patterns focus on how classes and objects are combined to form larger structures. They improve flexibility by managing relationships between objects.

Main Problems Solved:

- Simplify complex object relationships.
- Allow incompatible classes to work together.
- Improve object reuse and system flexibility.

Patterns:

1. Adapter Pattern

Purpose:

- Converts one interface into another interface expected by the client.
- Allows incompatible systems to communicate.

Usage:

- Integrating legacy systems.
- Connecting third-party libraries.

Advantages

The pros of Adapter Design Pattern:

- Promotes code reuse without modification.
- Keeps classes focused on core logic by isolating adaptation.
- Supports multiple interfaces through interchangeable adapters.
- **Decouples** system from implementations, easing modifications and swaps.

Disadvantages

The cons of Adapter Design Pattern:

- Adds complexity and can make code harder to follow.
- Multiple adapters increase maintenance effort.
- Handling many interfaces may require multiple adapters, complicating design.

2. Decorator Pattern

Purpose:

- Adds new functionality to an object dynamically without modifying its original class.
- Provides an alternative to creating many subclasses.

Usage:

- Adding pizza toppings.
- Adding extra features to user accounts.

Advantages

This pattern is useful when you want to add responsibilities to objects dynamically at runtime.

- Improves flexibility by allowing behavior to be added or removed at runtime.
- Promotes **Single Responsibility Principle** as functionality can be divided among classes.
- Avoids the use of large subclasses that combine multiple behaviors.

Disadvantages

- Can make the system more complex due to many small decorator classes.
- Debugging and understanding the flow of decorated objects may be harder.

3. Composite Pattern

Purpose:

- Treats individual objects and groups of objects in the same way.

- Represents objects using a tree structure.

Usage:

- File and folder systems.
- Organization hierarchy.

3. Behavioral Design Patterns

Behavioral patterns focus on communication between objects and distribution of responsibilities. They define how objects interact with each other.

Main Problems Solved:

- Manage object communication.
- Reduce dependency between objects.
- Make workflows and responsibilities easier to maintain.

Pattern:

1. Chain of Responsibility Pattern

Purpose:

- Passes a request through a chain of handlers until one handler processes it.
- Removes direct dependency between sender and receiver.

Usage:

- Approval systems.
- Event processing systems.

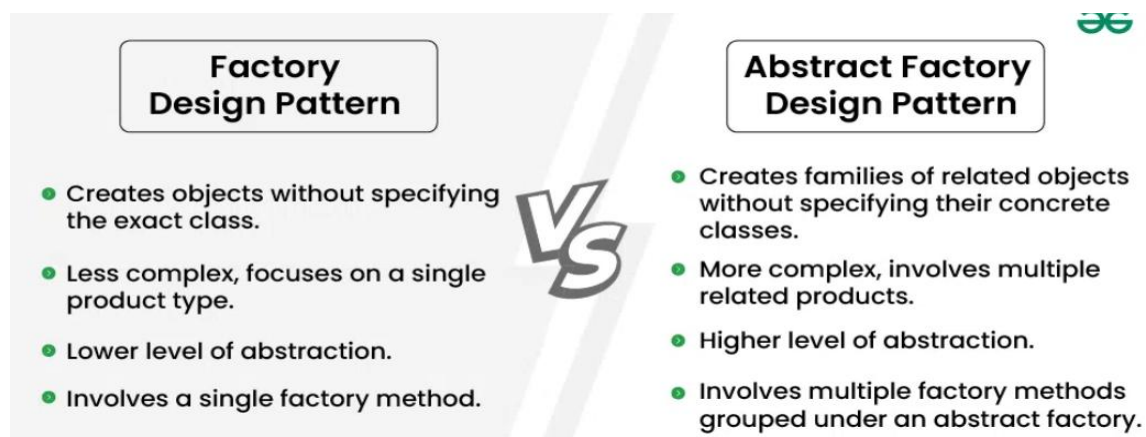
Advantages of Chain of Responsibility Pattern

1. **Reduces Coupling**
Sender does not need to know which object will handle the request.
2. **Improves Flexibility**
New handlers can be added or modified without changing client code.
3. **Simplifies Request Handling**
Separates request sending from processing logic.

Disadvantages of Chain of Responsibility Pattern

1. **Request May Not Be Handled**
If no handler can process the request, it may remain unresolved.
2. **Performance Issue**
A long chain may slow down request processing.
3. **Difficult Debugging**
Tracking which handler processed the request can be challenging.

Difference Between Factory and Abstract Factory Pattern



Advantages of Design Patterns

1. Reusability

Provides proven solutions that can be reused in different software projects.

Problem Solved: Reduces development time and avoids rewriting common solutions.

2. Flexibility

Allows software to adapt easily when requirements change.

Problem Solved: New features can be added with fewer modifications.

3. Maintainability

Creates organized and structured code.

Problem Solved: Makes debugging, updating, and extending software easier.

4. Reduced Coupling

Separates classes and responsibilities.

Problem Solved: Changes in one part of the system have less impact on other parts.

5. Better Scalability

Helps systems grow without becoming difficult to manage.

Problem Solved: Supports adding new classes, features, and behaviors.

6. Standard Communication

Provides common terminology among developers.

Problem Solved: Improves teamwork because developers can discuss solutions using pattern names.

Usage of Design Patterns

Design patterns are used as standard reusable solutions for common software design problems. They help developers build software that is flexible, maintainable, scalable, and easier to modify.

1. Managing Object Creation (Creational Patterns)

Creational patterns simplify and control how objects are created. They reduce dependency on concrete classes and make object creation more flexible.

Examples:

- **Singleton:** Ensures only one instance of a shared resource exists.
- **Factory:** Creates objects without exposing creation logic.
- **Builder:** Creates complex objects step by step.

2. Organizing Class and Object Structures (Structural Patterns)

Structural patterns define how classes and objects are combined to create flexible systems. They help integrate existing systems and improve object reusability.

Examples:

- **Adapter:** Connects incompatible interfaces.
- **Decorator:** Adds new features dynamically without changing existing classes.
- **Composite:** Organizes objects into tree structures.

3. Managing Object Communication (Behavioral Patterns)

Behavioral patterns define how objects communicate and distribute responsibilities. They simplify complex interactions and make system behavior easier to manage.

Examples:

- **Chain of Responsibility:** Passes requests through multiple handlers.

4. Supporting Refactoring and Removing Code Smells

Design patterns help improve existing code by replacing poor design structures with better solutions. They remove problems such as duplicate code, large conditional statements, and tightly coupled classes.

5. Implementing Good Design Principles

Design patterns help apply principles like **SOLID**, especially Open-Closed Principle and Dependency Inversion. They create loosely coupled, modular systems that are easier to extend, test, and maintain.

Note: Design patterns should be applied only when they solve an actual problem. Unnecessary use of patterns can increase complexity and make the system harder to understand.

Case Study: University HR System

- A university has several colleges. Each college has its own HR system and employees.

Topics:

1. Creating a single university instance
2. Creating employee objects based on designation
3. Converting different HR system formats into one HR system format
4. Teacher inherits from Employee class

Its gonna be like A university has multiple colleges, and each college has its own HR system and employee information. The system needs to combine all HR operations into a single university management system. Different design patterns are applied to solve different problems

1. Creating a Single University Instance

Pattern Used: Singleton Pattern

The university has common information such as university name, address, and registration details. This information should exist only once in the entire system. The Singleton Pattern ensures that only one University object is created and shared by all colleges and employees.

Problem Solved: Avoids creating multiple university objects and maintains consistent university information.

2. Creating Employee Objects Based on Designation

Pattern Used: Factory Pattern

Employees have different designations such as Teacher, Officer, and Staff. Instead of creating objects directly, a Factory class creates the appropriate employee object based on the designation provided.

Example:

Designation = Teacher → Create Teacher Object

Designation = Officer → Create Officer Object

Designation = Staff → Create Staff Object

Problem Solved: Reduces dependency on specific employee classes and simplifies object creation.

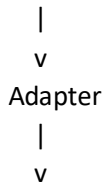
3. Converting Different HR System Formats into One HR Format

Pattern Used: Adapter Pattern

Each college may have its own HR system with different data formats and interfaces. The Adapter Pattern converts these different formats into a common HR system format so that all employee information can be processed uniformly.

Example:

College HR System 1



Common University HR System

Problem Solved: Allows different systems to work together without modifying existing HR systems.

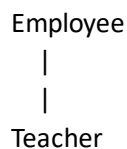
4. Teacher Inherits from Employee Class

Concept Used: Inheritance and Runtime Polymorphism

Teacher inherits common properties from the Employee class and adds teacher-specific features. Runtime polymorphism allows different employee types to provide their own implementation of methods like write_info().

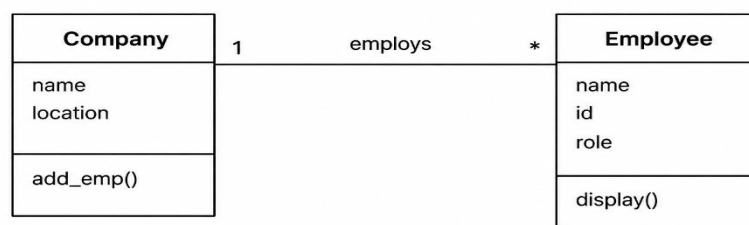
Problem Solved:

Reduces code duplication and allows flexible handling of different employee objects using a common interface.



Qualified Association

A **Qualified Association** in UML is a special type of association where an object is accessed using a **qualifier (key)**.



Pseudocode

Class Employee:

```
emp_id
name
```

Class Company:

```
employees = {}
```

```
addEmployee(employee):  
    employees[employee.emp_id] = employee
```

```
findEmployee(emp_id):  
    return employees[emp_id]
```

Main:

```
create Company
```

```
create Employee(101, "Rahim")  
add employee to Company
```

```
search Employee using emp_id
```

```
display Employee details
```